PROGRAMA DEL CURSO
IF7100 - INGENIERÍA DEL SOFTWARE
I CICLO 2026

1 Datos Generales

- Sigla: IF7100
- Nombre: Ingeniería del software
- Tipo de curso: Teórico-práctico
- Créditos: 4
- Horas lectivas: 7
- Desglose de las horas lectivas: 4 teoría, 3 práctica
- Requisitos: IF009 Desarrollo de software IV o IF-6100 Análisis de sistemas
- Correquisitos: No
- Ubicación en el plan de estudio: : Optativo, Área de tendencias de desarrollo de Software
- Suficiencia: No aplica
- Tutoría: Sí aplica
- Modalidad del curso: Presencial
- Clasificación: Propio

2 Descripción

El curso de Ingeniería de Software proporciona al estudiantado los fundamentos teóricos y prácticos necesarios para el desarrollo profesional de sistemas de software. A lo largo del curso, se abordarán conceptos esenciales, metodologías modernas, técnicas especializadas y herramientas CASE útiles en las diferentes etapas del ciclo de vida del software: desde el análisis y el diseño, hasta el desarrollo, pruebas, implementación y mantenimiento.

Se hace especial énfasis en el modelado arquitectónico del software y en la gestión de la calidad del producto mediante diversas estrategias de pruebas. El estudiantado participante aprenderán a determinar y analizar requerimientos de sistemas, definir estrategias de desarrollo efectivas y administrar el proceso de desarrollo de software de manera integral.

El componente práctico del curso se logra a través del desarrollo de un proyecto de software mediante el cual el estudiantado aplicará los conocimientos adquiridos en un escenario real, permitiéndole experimentar de primera mano, la innovación y la creatividad en el desarrollo de productos de software.

3 Resultados de aprendizaje del curso

Al finalizar el curso el estudiantado estará en capacidad de:

- Constrastar los modelos de proceso y las metodologías de desarrollo, en los diversos contextos de proyectos en los que pueden ser utilizados.
- Elicitar requerimientos de software a partir de la identificación de los involucrados e involucradas, sus necesidades y mediante el uso de diversas técnicas.
- Diseñar la arquitectura de software que satisfaga los requerimientos funcionales y no funcionales, mediante la aplicación de los patrones arquitectónicos, principios y compensaciones de diseño.
- Crear el diseño de alto nivel y el diseño detallado del software con base en los requerimientos, que sirva como base para el proceso de implementación.
- Construir el software con base en los requerimientos, la arquitectura y el diseño propuestos en acorde con las prácticas de las tecnologías de implementación.

- Explicar las actividades necesarias para operar un sistema de software mediante la entrega de sus servicios en un entorno operativo.
- Describir los procesos, actividades y técnicas asociadas al mantenimiento del software.

4 Contenidos

1. **Introducción a la ingeniería de software**
   - 1.1 ¿Qué es la ingeniería de software?
   - 1.2 Impulsores de los proyectos de software: clientes, procesos, productos y restricciones.
   - 1.3 Retos del desarrollo del software
2. **Procesos y metodologías de software**
   - 2.1 El proceso de software
   - 2.2 Modelos de procesos de software
     - 2.2.1 Cascada
     - 2.2.2 Proceso de prototipado
     - 2.2.3 Proceso evolutivo
     - 2.2.4 Proceso en espiral
     - 2.2.5 Basado en componentes
     - 2.2.6 Proceso unificado
     - 2.2.7 Procesos ágiles
   - 2.3 Diferencia entre proceso y metodología
   - 2.4 Metodologías ágiles
     - 2.4.1 Scrum
     - 2.4.2 Kanban
3. **Ingeniería de sistemas**
   - 3.1 ¿Qué es un sistema?
   - 3.2 ¿Qué es la ingeniería de sistemas?
   - 3.3 Introducción a las fases del proceso de la ingeniería en sistemas

3.3.1 Definición de los requerimientos del sistema
3.3.2 Diseño y modelado de sistemas
3.3.3 Desarrollo del hardware
3.3.4 Desarrollo del software
3.3.5 Desarrollo del talento humano
3.3.6 Pruebas e integración del sistema
3.3.7 Despliegue del sistema
3.3.8 Mantenimiento del sistema

4. **Requerimientos de software**

4.1 Fundamentos de ingeniería de requerimientos

4.1.1 ¿Qué es la ingeniería de requerimientos?
4.1.2 Definición de requerimientos de software
4.1.3 Requerimientos funcionales vs. no funcionales
4.1.4 Desafíos e importancia de la elicitación de requerimientos

4.2 Identificación de requerimientos

4.2.1 Técnicas para recopilar información: encuestas, entrevistas, análisis de documentos, revisiones de literatura, observación, talleres

4.3 Especificación de requerimientos

4.3.1 Documentación de requerimientos
4.3.2 Especificación de requerimientos de Software (ERS)
4.3.3 Análisis y priorización de requerimientos: identificación de necesidades, métodos de estimación de esfuerzos, el método MOSCOW

4.4 Modelado y especificación de casos de uso

4.4.1 Conceptos: Actores, Casos de Uso, Límites del Sistema, Escenarios
4.4.2 Diagramas de Casos de Uso: Notación y Uso
4.4.3 Especificación de Casos de Uso
4.4.4 Tipos de Casos de Uso: Abstracto, de Alto Nivel, Expandido

4.5 Historias usuarias

4.5.1 Pasos para la definición de historia usuaria (3 C): tarjeta, conversación y confirmación
4.5.2 Escritura de la historia usuaria

4.5.3 Escritura de los criterios de aceptación
4.5.3.1 Criterios de aceptación orientados al escenario
4.5.3.2 Criterios de aceptación orientados a las reglas
4.6 Validación de requisitos
4.7 Administración de requerimientos
4.7.1 Trazabilidad de requisitos y gestión de cambios.
4.7.2 Herramientas de administración de requisitos.

5. **Modelado de dominio**

5.1 Conceptos y principios del modelado de dominio
5.2 Técnicas para identificar conceptos de dominio: análisis sustantivo-verbal, análisis de casos de uso
5.3 Clasificación de conceptos de dominio: Entidades, atributos, relaciones, restricciones
5.4 Diagramas de clases UML: Notación y semántica
5.5 Refinamiento y validación del modelo de dominio

6. **Documentación y modelado de procesos organizacionales**

6.1 Conceptos: proceso, características de los procesos
6.2 Tipos de procesos: operativos, de apoyo y de gestión
6.3 Formas alternativas para documentar los procesos
6.4 Relación de los procesos con la ingeniería de software
6.5 Técnicas para la definición y diseño de procesos: diagramas de actividad UML y diagramas de flujos de proceso (flujogramas)
6.6 Notación y simbología utilizada para modelado de procesos
6.7 Herramientas CASE para el modelado de procesos

7. **Arquitectura del Software**

7.1 Introducción a la arquitectura de software
7.1.1 El concepto de arquitectura de software
7.1.2 Espectro entre arquitectura y diseño
7.1.3 Estrategia vs táctica

7.1.4 Esfuerzo alto vs esfuerzo bajo
7.1.5 Relación de la Arquitectura de Software con otras áreas de la Ingeniería de Software
7.2 Dimensiones de la arquitectura de software
7.2.1 Características arquitectónicas
7.2.2 Decisiones arquitectónicas
7.2.2.1 Compensaciones (Trade-offs)
7.2.2.2 Primera ley de la arquitectura de software: Todo es una compensación
7.2.2.3 Segunda ley de la arquitectura de software: El porqué es más importante que el cómo
7.2.3 Consideraciones para la toma de decisiones arquitectónicas
7.2.3.1 Registros Arquitectónicos de decisiones (ADRs)
7.2.3.2 Fitness Functions
7.2.4 Componentes lógicos
7.2.4.1 Arquitectura lógica vs arquitectura física
7.2.4.2 Creación de una arquitectura lógica
7.3 Estilos Arquitectónicos
7.3.1 Clasificación de tipos de estilos arquitectónicos: Modelo de despliegue y partición
7.3.2 Ventajas y desventajas de los modelos de despliegue: Monolito y Distribuido
7.3.3 Ventajas y desventajas de los modelos de partición: Técnicas y Dominio
7.3.4 Diseño dirigido por capas
7.3.4.1 MVC: Aplicación de capas en diferentes tecnologías
7.3.5 Diseño dirigido por el dominio (DDD)
7.3.6 Monolitos modulares
7.3.7 Microkernel
7.3.8 Microservicios
7.4 Diagramación y documentación de la arquitectura de software
7.4.1 El modelo C4
7.4.2 Diagramación UML: componentes, paquetes, despliegue

8. **Diseño del software**

   8.1 Introducción

      8.1.1 Conceptos de diseño

      8.1.2 Interacciones entre el diseño y los requisitos

      8.1.3 Principios de diseño: acoplamiento, cohesión, SOLID

   8.2 Estrategias de diseño

   8.3 Diseño orientado a objetos

   8.4 Diseño orientado a datos

   8.5 Trazabilidad de requisitos en la arquitectura

   8.6 Patrones de diseño

      8.6.1 Creacionales

      8.6.2 Estructurales

      8.6.3 Comportamiento

   8.7 Modelado y diseño de sistemas interactivos

      8.7.1 Modelado de interacción de objetos: diagrama de secuencia

      8.7.2 Modelos de comportamiento: diagramas de máquina de estado y de estado.

   8.8 Modelado y diseño de sistemas impulsados por eventos

   8.9 Experiencia de usuario (UI/UX)

      8.9.1 Técnicas diseño visual para UX

         8.9.1.1 Diseño inclusivo

         8.9.1.2 Interfaces de voz

         8.9.1.3 Diseño Mobile-First

      8.9.2 Modalidades de interfaz

      8.9.3 Patrones de diseño UX

      8.9.4 Tendencias de diseño Web

         8.9.4.1 User Experience

         8.9.4.2 User Interface

         8.9.4.3 Las heurísticas de Nielsen

   8.10 Evaluación de diseño

9. **Construcción del software**

- 9.1 Fundamentos
  - 9.1.1 Principios fundamentales de la construcción
    - 9.1.1.1 Modularidad, Reutilización, Verificabilidad, Adaptabilidad
  - 9.1.2 Técnicas para minimizar la complejidad del software
    - 9.1.2.1 Principios de diseño (KISS, DRY, YAGNI, SOLID)
  - 9.1.3 Estándares de documentación y codificación.
    - 9.1.3.1 Importancia y aplicación de estándares de codificación.
- 9.2 Gestión de la Construcción
  - 9.2.1 Planificación del proyecto y gestión de tareas
    - 9.2.1.1 Definición de tareas y asignación de recursos
    - 9.2.1.2 Herramientas de Planificación: PERT, Gantt, Planeación ágil
  - 9.2.2 Gestión de Riesgos
    - 9.2.2.1 Identificación de riesgos
    - 9.2.2.2 Análisis de riesgos
  - 9.2.3 Métodos de medición
    - 9.2.3.1 Líneas de Código (LOC)
    - 9.2.3.2 Complejidad Ciclomática
    - 9.2.3.3 Puntos de Función
  - 9.2.4 Estimación del esfuerzo
    - 9.2.4.1 Modelo COCOMO II
    - 9.2.4.2 Estimación en metodologías ágiles
- 9.3 Pruebas unitarias e integración en el proceso de construcción.

10. **Operaciones de la ingeniería de software**

- 10.1 Fundamentos de DevOps:
  - 10.1.1 ¿Qué es DevOps?
  - 10.1.2 Definición y evolución
  - 10.1.3 Cultura DevOps
  - 10.1.4 Beneficios y desafíos
  - 10.1.5 Principios básicos
  - 10.1.6 Automatización
  - 10.1.7 Medición continua

- 10.1.8 Compartir conocimientos
- 10.1.9 Retroalimentación rápida

10.2 Automatización y CI/CD:

- 10.2.1 Integración Continua (CI)

10.3 Entrega y despliegue continuo

- 10.3.1 Continuous Delivery/Deployment
- 10.3.2 Estrategias de despliegue
  - 10.3.2.1 Blue-Green deployments
  - 10.3.2.2 Canary releases
- 10.3.3 Contenedores y Orquestación

10.4 Infraestructura y Gestión de Configuración:

- 10.4.1 Infraestructura como Código
- 10.4.2 Cloud Computing

11. **Mantenimiento del software**

11.1 Conceptos básicos de mantenimiento del software

- 11.1.1 Necesidad de mantenimiento en el ciclo de vida del software.
- 11.1.2 Importancia y objetivos del mantenimiento.
- 11.1.3 Costos asociados y su impacto a largo plazo en los proyectos.

11.2 Tipos de mantenimiento del software

- 11.2.1 Correctivo
- 11.2.2 Adaptativo
- 11.2.3 Perfectivo
- 11.2.4 Preventivo

11.3 Procesos y modelos de mantenimiento

- 11.3.1 Introducción al ciclo de vida del mantenimiento del software (SMLC).
- 11.3.2 Transición de desarrollo a mantenimiento: desafíos y prácticas recomendadas.
- 11.3.3 Análisis de solicitudes de cambio y toma de decisiones sobre modificaciones.

11.4 Estrategias de implementación

- 11.4.1 Planificación de lanzamientos de mantenimiento.

11.4.2 Gestión de la deuda técnica y estrategias de refactorización.

11.4.3 Uso de herramientas de gestión de configuración para mantener la integridad del sistema.

11.5 Retos en el mantenimiento del software

11.5.1 Impacto de los cambios en la calidad del software (ripple effects y bugs introducidos).

11.5.2 Dificultades en la comprensión del código heredado y prácticas para mejorarla.

11.5.3 Problemas asociados con sistemas obsoletos (legacy systems).

11.6 Pruebas y validación durante el mantenimiento

11.6.1 Estrategias de pruebas de regresión y validación.

11.6.2 Automatización de pruebas y su rol en la estabilidad del software.

11.7 Ingeniería inversa y reingeniería de software

5 Metodología

Las actividades del curso se apoyarán en diversas Tecnologías de Información y Comunicación (TIC), utilizando como plataforma principal el entorno oficial de Mediación Virtual de la Universidad de Costa Rica, a través de la cuenta de correo electrónico institucional (ucr.ac.cr). Este entorno servirá como medio de interacción entre la persona docente y el estudiantado, así como repositorio de información, espacio para la entrega de trabajos extraclase y aplicación de evaluaciones. Todos los cursos estarán regidos por las resoluciones, lineamientos y estrategias activas de la Universidad de Costa Rica. (VD-R-8458-2009, VD-R-9374-2016 y VD-12784-2023).

Adicionalmente, se podrán utilizar otras herramientas como Microsoft Teams y Zoom, a las cuales se tiene acceso por medio de la cuenta de correo institucional, así como plataformas no oficiales, por ejemplo, WhatsApp, Telegram, entre otras. El objetivo es ampliar las posibilidades de comunicación y participación de toda la población estudiantil, ofreciendo múltiples opciones de conexión e interacción que favorezcan una educación más accesible.

Los contenidos del curso podrán ser abordados con estrategias en concordancia con la modalidad que se indica en la parte superior de este programa, fomentando la capacidad analítica y posibilitando la aclaración de dudas y la corrección de errores.

En este contexto, los errores que se puedan cometer serán considerados como fuente y oportunidad de aprendizaje.

Independientemente de la modalidad del curso, se promoverá una participación activa de la persona estudiante en todas las sesiones. Se fomentarán valores fundamentales como la responsabilidad, la disciplina, la integridad, la equidad, la excelencia y la perseverancia. En este sentido, se espera que las personas estudiantes asuman un rol protagónico en su proceso formativo, reflexionando críticamente sobre los contenidos y cumpliendo con las tareas asignadas de manera oportuna y rigurosa.

Durante el desarrollo del curso, el personal docente desempeñará un rol activo en la exposición de los contenidos fundamentales mediante presentaciones magistrales y el uso de diversas estrategias didácticas orientadas a promover una comprensión profunda y significativa de los temas abordados. Estas estrategias podrían incluir foros de discusión, investigaciones, resolución de casos, debates guiados, actividades extracurriculares y dinámicas grupales, entre otras, las cuales permiten a las personas estudiantes vincular los conceptos teóricos con situaciones del mundo real. Asimismo, se podrá fomentar la participación estudiantil a través de presentaciones individuales o grupales, complementadas con el análisis crítico de lecturas y estudios de investigación, lo que contribuirá al desarrollo de habilidades analíticas y argumentativas en un entorno de aprendizaje colaborativo. Para enriquecer este proceso, el curso podrá incorporar el uso de software especializado y medios audiovisuales, facilitando así una experiencia de aprendizaje más dinámica, interactiva y contextualizada.

El personal docente y la población estudiantil desarrollarán las clases dentro de un ambiente de tolerancia, respeto y comunicación asertiva. El profesorado promoverá el trabajo en equipo, en un plano de igualdad de oportunidades y sin discriminación de ninguna especie de forma tal que se garantice un ambiente de diálogo y libre expresión de las ideas y opiniones.

5.1 Estrategias de evaluación

Este curso se desarrollará bajo un enfoque teórico-práctico, integrando la exposición de contenidos conceptuales con su aplicación en contextos reales y simulados. Se promoverá el aprendizaje activo mediante el análisis, la experimentación, la resolución de casos y la reflexión crítica, con el fin de fortalecer las competencias técnicas y profesionales del estudiantado en el campo de la informática.

El proceso de evaluación será integral, continuo y formativo, y se llevará a cabo mediante diversos instrumentos que permiten valorar tanto el dominio conceptual como la capacidad de aplicación práctica. A continuación, se describen los tipos de evaluación que se podrían utilizar en el curso:

- **Exámenes teóricos:** Son pruebas escritas estructuradas que evalúan la comprensión de conceptos, teorías, definiciones, metodologías y marcos de referencia. Pueden incluir preguntas de opción múltiple, verdadero/falso, desarrollo, análisis de casos, ensayos breves, entre otros. Su propósito es medir la capacidad del estudiantado para razonar, argumentar y explicar los contenidos del curso.

- **Exámenes prácticos:** Estas evaluaciones se pueden orientar en la resolución de problemas técnicos mediante el uso de herramientas, lenguajes de programación, entornos de desarrollo o plataformas específicas. Se valorará la precisión, eficiencia, lógica y creatividad en la solución propuesta, así como la capacidad para interpretar requerimientos y aplicar buenas prácticas.

- **Exámenes teótico-prácticos:** Combinan elementos de evaluación conceptual con ejercicios de aplicación. Por ejemplo, pueden incluir una sección de preguntas teóricas seguida de una actividad práctica que requiera implementar o analizar una solución. Este tipo de examen busca integrar el conocimiento con la acción.

- **Quices:** Son pruebas breves que pueden aplicarse con el objetivo de reforzar el repaso constante de los contenidos y verificar el seguimiento del curso. Generalmente se enfocan en temas recién abordados.

- **Tareas:** Son actividades asignadas para realizar fuera del horario de clase, que permiten profundizar en los temas tratados, desarrollar habilidades de investigación, análisis y síntesis, y fomentar la autonomía del aprendizaje. Pueden incluir ejercicios, resúmenes, mapas conceptuales, análisis de lecturas, resolución de casos, entre otros.

- **Proyectos programados** Consisten en el desarrollo de soluciones informáticas, investigaciones aplicadas o productos tecnológicos que integran múltiples contenidos del curso. Se valorará la planificación, ejecución, documentación, presentación y defensa del proyecto.

- **Prácticas guiadas:** Son sesiones prácticas estructuradas en las que el estudiantado aplica los conocimientos adquiridos bajo la orientación de la persona

docente. Estas prácticas permiten afianzar habilidades técnicas, resolver dudas en tiempo real y recibir retroalimentación de la persona docente.

- **Investigaciones:** Se asignarán temas específicos para ser investigados de forma individual o grupal, con el fin de fomentar el pensamiento crítico, la búsqueda de información confiable, la elaboración de argumentos propios y la presentación de resultados de manera clara y fundamentada.

- **Foros:** Son espacios de discusión asincrónica o sincrónica, presenciales o virtuales, donde se promueve el intercambio de ideas, la argumentación respetuosa y el análisis crítico de temas relevantes. La participación activa, la calidad de las intervenciones y la capacidad de escucha serán criterios de evaluación.

- **Resolución de casos:** Es una metodología que permite a la persona estudiante aplicar sus conocimientos en informática mediante el análisis y solución de situaciones reales o simuladas. A través de esta estrategia, se fomenta el pensamiento crítico, la toma de decisiones y el trabajo colaborativo, al tiempo que se desarrollan habilidades técnicas y comunicativas. Las personas estudiantes analizan un problema, proponen soluciones viables y, en algunos casos, implementan un prototipo, fortaleciendo así su preparación para enfrentar desafíos del entorno profesional.

- **Taller:** Consiste en que las personas estudiantes trabajen en ejercicios, proyectos o desafíos relacionados con los contenidos del curso, bajo la guía de la persona docente. Aunque puede incluir momentos de explicación, el énfasis está en el "aprender haciendo", lo que favorece la comprensión profunda y significativa de los temas abordados. Esta metodología también permite una retroalimentación inmediata, facilitando la corrección de errores y el refuerzo de buenas prácticas.

- **Actividades extracurriculares:** Son actividades como: ferias académicas, charlas, talleres, simposios, giras, proyectos sociales, intercambios, congresos, seminarios, entre otras. Estas actividades deben de contribuir al logro de los resultados de aprendizaje del curso, por lo tanto, podrán ser consideradas como parte de la evaluación.

- **Lecturas en inglés:** Para apoyar los temas desarrollados en clase, se incluirán al menos dos lecturas en inglés. El objetivo principal de esta práctica es fomentar la comprensión lectora en dicho idioma, una competencia clave en el ámbito de la informática. Dado que el nivel lingüístico del estudiantado puede

variar, los reportes y presentaciones sobre el material leído deberán realizarse en español, asegurando así la equidad en la evaluación. La persona docente indicará oportunamente al estudiantado la lectura asignada, así como la fecha de realización de la actividad.

6 Evaluación

DESCRIPCIÓN	PORCENTAJE
Examen parcial I	20%
Examen parcial II	20%
Prácticas guiadas	15%
Proyecto práctico	35%
Investigación	10%
TOTAL	**100%**

6.1 Consideraciones sobre la evaluación y el curso

El curso utilizará la plataforma oficial de la Universidad de Costa Rica, Mediación Virtual (https://mv.mediacionvirtual.ucr.ac.cr/). A través de este espacio, el personal docente compartirá el programa del curso, materiales de apoyo, enunciados de evaluaciones y otros recursos relevantes. El estudiantado, por su parte, deberá utilizar esta plataforma para enviar entregables y/o realizar evaluaciones asignadas.

Es responsabilidad del estudiantado mantener activo su correo institucional, revisarlo con regularidad y responder oportunamente a las comunicaciones recibidas por este medio.

Los criterios de evaluación de cada actividad serán detallados por parte de la persona docente del curso mediante rúbricas específicas, las cuales estarán disponibles en los respectivos enunciados de evaluación.

6.2 Consideraciones académicas y éticas

Se fomentará la honestidad académica y el cumplimiento ético en todas las actividades del curso, promoviendo la prevención del plagio. En caso de detectarse una

falta, como copia o plagio, ya sea total o parcial, realizada por la persona estudiante con el apoyo de terceros o mediante el uso indebido de herramientas de inteligencia artificial no autorizadas, se procederá de acuerdo con lo estipulado en el Reglamento de Orden y Disciplina Estudiantil.

6.3 Comunicación y reprogramación de evaluaciones

Toda evaluación será comunicada al estudiantado al menos con cinco días hábiles de antelación, con excepción de las pruebas cortas o "quices", conforme a lo especificado en los artículos 15 y 18 del Reglamento de Régimen Académico Estudiantil. En caso de ausencia a una evaluación, se procederá según lo estipulado en el Artículo 24 del mismo reglamento.

6.4 Flexibilidad y accesibilidad

Las fechas del cronograma están sujetas a cambios dependiendo del avance en los contenidos del curso y de imprevistos (propios y ajenos al curso y de la institución) durante el desarrollo del semestre, asegurando siempre desarrollar todos los contenidos indicados en este programa.

Todas las evaluaciones deben garantizar la accesibilidad para el estudiantado con necesidades educativas especiales. En estos casos, la persona estudiante deberá gestionar y coordinar ante las instancias respectivas.

6.5 Actividades extracurriculares

En el caso que se realicen actividades extracurriculares, como las mencionadas en el apartado de Estrategias de evaluación, la persona docente podrá considerar asignar valor porcentual, siempre que se justifique su pertinencia y se establezcan criterios claros de valoración, quedando establecida como actividades del curso, en el rubro de pruebas cortas.

7 Docentes del curso

GRUPO	DOCENTE	HORARIO	CONSULTA
	SEDE DEL ATLÁNTICO, RECINTO DE GUÁPILES		
31	Lic. Arturo Guillén Berrocal geber.guillen@ucr.ac.cr	K: 17 a 20:50 J: 17 a 19:50	S: 16 a 19:30
	SEDE DEL ATLÁNTICO, RECINTO DE PARAÍSO		
01	MSc.José Camacho Navarro jose.camacho@ucr.ac.cr	K: 17 a 20:50 V: 17 a 19:50	J: 17 a 20:30
	SEDE DEL ATLÁNTICO, RECINTO DE TURRIALBA		
01	Mag. Juan José Quesada Sánchez juanjose.quesada@ucr.ac.cr	K: 19 a 20:50 S: 07 a 11:50	V: 18 a 20:00 S: 13 a 14:30
	SEDE DEL CARIBE, RECINTO DE LIMÓN		
01	MDS. Rodrigo Torres Sanchez rodalonts@gmail.com	S: 9 a 11:50 S: 13 a 16:50	J: 19 a 21:30 S: 8 a 9:00
	SEDE DEL CARIBE, AULA DE SIQUIRRES		
01	Lic. Delia Elvira Smith Paul delia.smith@ucr.ac.cr	L: 08 a 11:50 M: 09 a 11:50	J: 13 a 16:30
	SEDE DE GUANACASTE, RECINTO DE LIBERIA		
01	Lic. Alonso Chavarria Cubero ivan.chavarriacubero@ucr.ac.cr	M: 13 a 16:50 V: 13 a 15:50	M: 8 a 11:30
	SEDE DE OCCIDENTE, RECINTO DE GRECIA		
01	MAP. Verny Fernández Castro verny.fernandez@ucr.ac.cr	K: 17 a 20:50 J: 17 a 19:50	J: 19 a 21:00 S: 8 a 9:30

GRUPO	DOCENTE	HORARIO	CONSULTA
	SEDE DE OCCIDENTE, RECINTO DE SAN RAMÓN		
01	MCI. Juan Carlos Miranda Sánchez juancarlos.miranda@ucr.ac.cr	L: 16 a 20:50 K: 7 a 08:50	L: 16 a 17:00 K: 9 a 11:30
	SEDE DEL PACÍFICO		
01	MS.c Raquel Porras Soto raquel.porrassoto@ucr.ac.cr	L: 8 a 11:50 V: 8 a 10:50	J: 8 a 11:30
	SEDE DEL SUR		
01	MS.c María José Peralta Varela maria.peraltavarela@ucr.ac.cr	K: 18 a 20:50 S: 8 a 11:50	S: 13 a 16:30

8 Cronograma

SEM	FECHA	TEMA O ACTIVIDAD
01	09 - 14 MAR	Lectura del programa del curso Socialización de información Institucional sobre Hostigamiento Sexual Tema 1. Introducción a la ingeniería de software
02	16 - 21 MAR	Tema 2. Procesos y metodologías de software
03	23 MAR - 28 MAR	Tema 3. Ingeniería de sistemas Tema 4. Requerimientos de software
04	30 MAR - 4 ABR	Semana Santa.
05	13 - 18 ABR	Tema 4. Requerimientos de software
06	20 - 25 ABR	Semana Universitaria. Tema 5. Modelado de Dominio
07	27 ABR - 2 MAY	Tema 5. Modelado de Dominio Tema 6. Documentación y modelado de procesos organizacionales

SEM	FECHA	TEMA O ACTIVIDAD
08	4 - 9 MAY	**Examen 1** Tema 7. Arquitectura del Software
09	11 - 16 MAY	Tema 7. Arquitectura del Software
10	18 - 23 MAY	Tema 8. Diseño del software
11	25 - 30 MAY	Tema 9. Diseño del software
12	1 - 6 JUN	Tema 10. Construcción del software
13	8 - 13 JUN	Tema 10. Construcción del software Tema 11. Operaciones de la ingeniería de software
14	15 - 20 JUN	Tema 11. Operaciones de la ingeniería de software
15	22 -27 JUN	Entrega de Proyecto Final. Tema 12. Mantenimiento del software
16	29 JUN - 4 JUL	Entrega de Proyecto Final. Tema 12. Mantenimiento del software **Examen 2**
17	6 - 11 JUL	Ampliación

9 Acreditación

La Carrera Bachillerato en Informática Empresarial está acreditada por el Sistema Nacional de Acreditación de la Educación Superior (SINAES) en el periodo comprendido entre el 10 de diciembre del 2019 al 3 de diciembre del 2023 (ACUERDO-CNA-400-2019) en las siguientes Sedes y Recintos:

- Sede Regional del Atlántico, Recinto de Guápiles
- Sede Regional del Atlántico, Recinto de Paraíso
- Sede Regional del Atlántico, Recinto de Turrialba
- Sede Regional del Caribe, Recinto de Limón
- Sede Regional de Guanacaste, Recinto de Liberia
- Sede Regional de Occidente, Recinto de Grecia
- Sede Regional de Occidente, Recinto de San Ramón
- Sede Regional del Pacífico



Referencias obligatorias

Ahmed, M. I. (2024). *Cloud-native devops: Building scalable and reliable applications*. Apress. Retrieved from https://springerlink.proxyucr.elogim.com/book/10.1007/979-8-8688-0407-6

Ambily, K. K. (2020). *Azure devops for web developers: Streamlined application development using azure devops features*. Apress. Retrieved from https://springerlink.proxyucr.elogim.com/book/10.1007/978-1-4842-6412-6

Brown, S. (2024). *The c4 model for visualising software architecture.* Retrieved from https://c4model.com/ (Accessed: 2024-12-06)

Dennis, A., Wixom, B. H., & Tegarden, D. (2021). *Systems analysis & design: An object-oriented approach with uml* (Sixth ed.). Wiley.

Ford, N., & Richards, M. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Gandhi, R., Richards, M., & Ford, N. (2024). *Head first software architecture: A learner's guide to architectural thinking*. O'Reilly Media.

IEEE Computer Society. (2024). *Guide to the software engineering body of knowledge v4.0* (Tech. Rep.). Author. Retrieved from https://www.swebok.org

Kaiser, A. K. (2023). *Reinventing ITIL® and DevOps with Digital Transformation: Essential Guidance to Accelerate the Process*. Apress. Retrieved from https://springerlink.proxyucr.elogim.com/book/10.1007/978-1-4842-9072-9

Laplante, P. A. (2022). *Requirements engineering for software and systems* (4th ed.). CRC Press.

Pressman, R. S., & Maxin, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). Mc Graw Hill.

Sneh Pandya, R. G. T. (2022). *Introduction to infrastructure as code: A brief guide to the future of devops*. Apress. Retrieved from https://springerlink.proxyucr.elogim.com/book/10.1007/978-1-4842-8689-0

Referencias secundarias

Dathan, B., & Ramnath, S. (2025). *Object-oriented analysis, design and implementation: An integrated approach* (3rd ed.). Springer Cham.

Farley, D. (2021). *Continuous delivery pipelines: How to build better software faster*. Leanpub.

Ford, N., Parsons, R., & Kua, P. (2017). *Building evolutionary architectures: Support constant change*. O'Reilly Media, Inc.

Fowler, M. (1997). *Uml distilled: A brief guide to the standard object modeling language*. Retrieved from `http://ci.nii.ac.jp/ncid/BA65029497`

Larman, C. (2004). *Applying uml and patterns: An introduction to object-oriented analysis and design and iterative development* (3rd Edition ed.). Prentice Hall PTR eBooks. Retrieved from `http://opac.lib.itenas.ac.id/index.php?p=show`$_d etailid = 11060 keywords =$

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Sundaramoorthy, S. (2022). *Uml diagramming: A case study approach*. CRC Press.

Winters, T., Manshreck, T., & Wright, H. (2022). *Ingeniería de software en google: Lecciones sobre programación aprendidas a lo largo del tiempo*. Marcombo.